在线商店系统架构与项目反思

Application Architecture IMAT3701
基线 4bfdc48 及本次质量改进 2026 年 10 月 29 日

OnlineStore 以 ASP.NET Core MVC 实现商品目录、购物车、订单和后台管理。当前应用由 Web、Business、DataAccess、Models 四个项目构成，仍部署为一个进程。本报告沿请求和数据的流向说明结构，并把可用性、可修改性、可测试性、易用性和安全性联系到具体故障、代码与验证证据。

一 系统架构 System Architecture

1.1 技术栈总览

技术栈：ASP.NET Core 8 MVC、C# 与 Razor。OnlineStore 是启动项目；OnlineStore.Business、OnlineStore.DataAccess、OnlineStore.Models 为三个类库；OnlineStore.Tests 独立执行测试。类库拆分建立编译依赖边界，没有形成微服务或四个独立部署单元。[S1]

数据访问：EF Core 8.0.11 配合 SQL Server。ApplicationDbContext 和迁移位于 DataAccess，启动配置显式指定迁移程序集。数据库连接由配置提供；验证工具使用独立环境变量与新建临时库，不复用日常 OnlineStore 数据库。[S1、S2、S8]

认证授权：ASP.NET Core Identity 处理账户、密码和角色。用户统一从 /Account/Login 登录，后台控制器要求 Admin 角色；订单和评价操作还校验用户与资源的归属。

状态与文件：SessionCartStore 将购物车 JSON 保存在服务器 Session，当前进程内缓存的空闲超时为 30 分钟。上传图片存入 wwwroot/uploads；两者都需要在重启、持久化和多实例部署时单独设计。

前端与外部服务：Razor、Bootstrap 和功能脚本组织双语目录、规格、订单与后台页面。邮件通过 IEmailService 使用配置选定的 Brevo HTTPS API 或 SMTP 实现；支付、资金退款与承运商 API 尚未真实接入。

资源与模型：Models 保存实体、视图模型、SharedResource 和双语资源。L10n 在英文内容缺失时回退；资源定位测试检查类库迁移后仍能取得中文和英文文本。共享模型有利于复用，也意味着字段变更仍需协调调用方。

验证分为纯规则与 .NET 回归、Node 前端行为、真实 SQL Server 集成以及运行时 HTTP 检查。2026 年 10 月 29 日的 Release 验证中，261 项 .NET 测试与 42 项 Node 测试全部通过；SQL 工具的 17 项隔离检查也全部通过。 各类测试的环境与局限在第 2.3 节和证据索引中说明。[S8]

1.2 项目依赖与运行边界

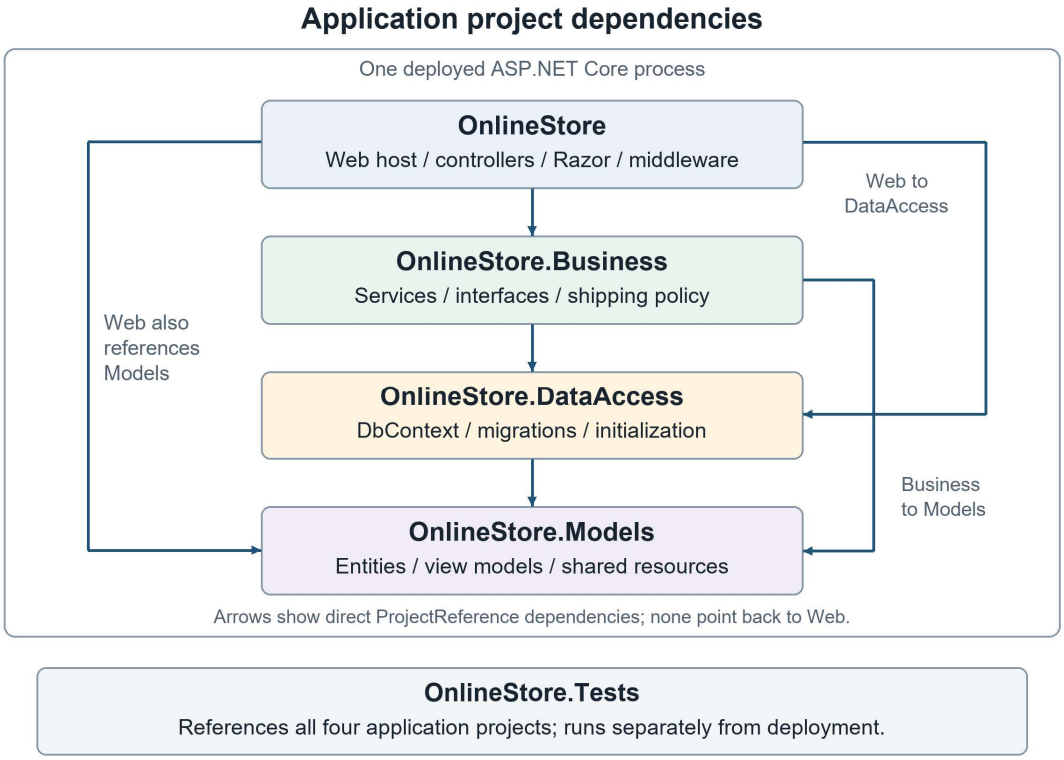


图 1 四个应用项目的直接依赖与单进程边界

图 1 的箭头表示 csproj 中的直接 ProjectReference。Web 引用三个类库；Business 引用 DataAccess 与 Models；DataAccess 只引用 Models；三个类库均不反向引用 Web。Tests 引用四个应用项目，作为独立验证入口。LayerBoundaryTests 检查程序集依赖，避免代码移动后再次引入反向耦合。[S1、S8]

Web 负责启动、依赖注入、HTTP 中间件、Controllers、Razor Views、ViewComponents 及静态资源。Areas/Admin 只是同一应用内的后台路由和组织方式。账号控制器仍直接调用 Identity，不能声称全部流程都严格经过自定义业务接口。

Business 集中商品、购物车、订单和轮播服务。本轮把下单物流规则抽为 IOrderShippingPolicy，接受不可变事实并返回报价；OrderService 继续拥有库存、优惠券与事务。该边界允许替换收费规则，同时避免多个服务争夺同一事务的提交权。[S4]

DataAccess 包含继承 IdentityDbContext 的 ApplicationDbContext、EF 映射、迁移和 DbInitializer。启动时执行迁移和受环境约束的种子初始化；它们不是每次请求的操作。Models 不引用业务或数据库项目，但仍引用共享 ASP.NET Core 框架，不能称为完全无框架依赖的领域核心。[S2]

运行时依赖包括 SQL Server、邮件服务、进程内 Session、上传目录和 Data Protection 密钥。图 1 展示编译依赖；数据库和邮件属于运行时依赖，不能由类库引用图推断其网络部署位置。

四个应用程序集在同一 Web 进程内加载。本次没有引入负载均衡、SQL 故障转移、跨实例限流或共享 Session。新增健康探针与故障反馈用于识别和控制故障影响，不能替代冗余部署。

1.3 MVC 模式

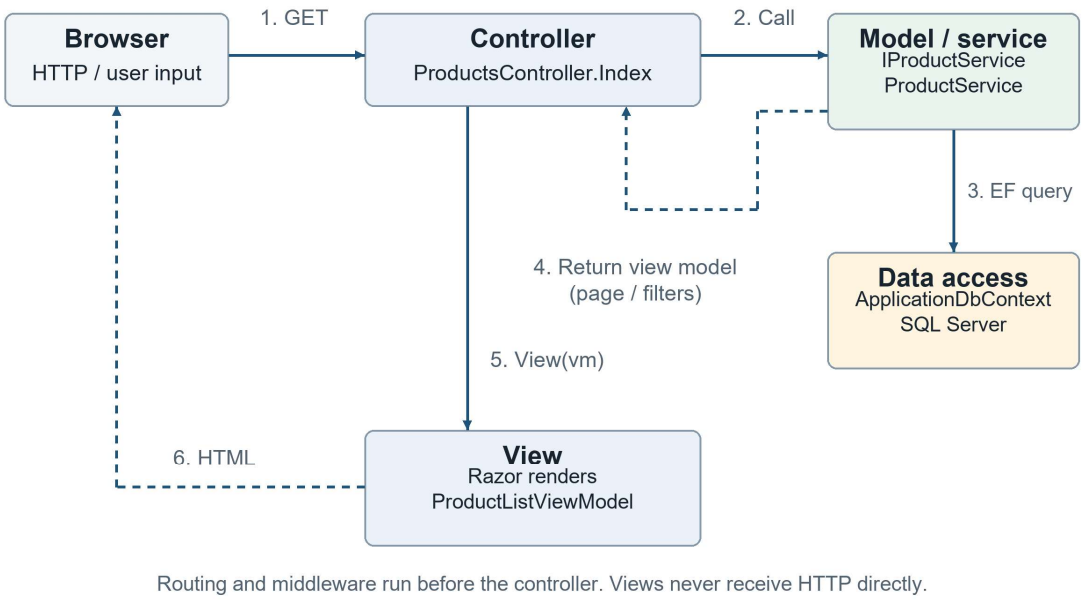


图 2 商品目录的 MVC 协作和响应路径

MVC（Model View Controller，模型、视图与控制器）在 Web 层区分请求编排与呈现。目录请求先经过中间件和路由，再由 ProductsController.Index 调用 IProductService.GetPagedAsync；服务经 DbContext 查询并组装 ProductListViewModel，控制器选择 Razor 视图，最终返回 HTML。Model 包含业务状态及协作，不只是数据库表；View 不直接接收 HTTP。项目分层与 MVC 是不同维度：一个控制器仍会跨类库调用服务。[S3]

1.4 成功请求与失败恢复

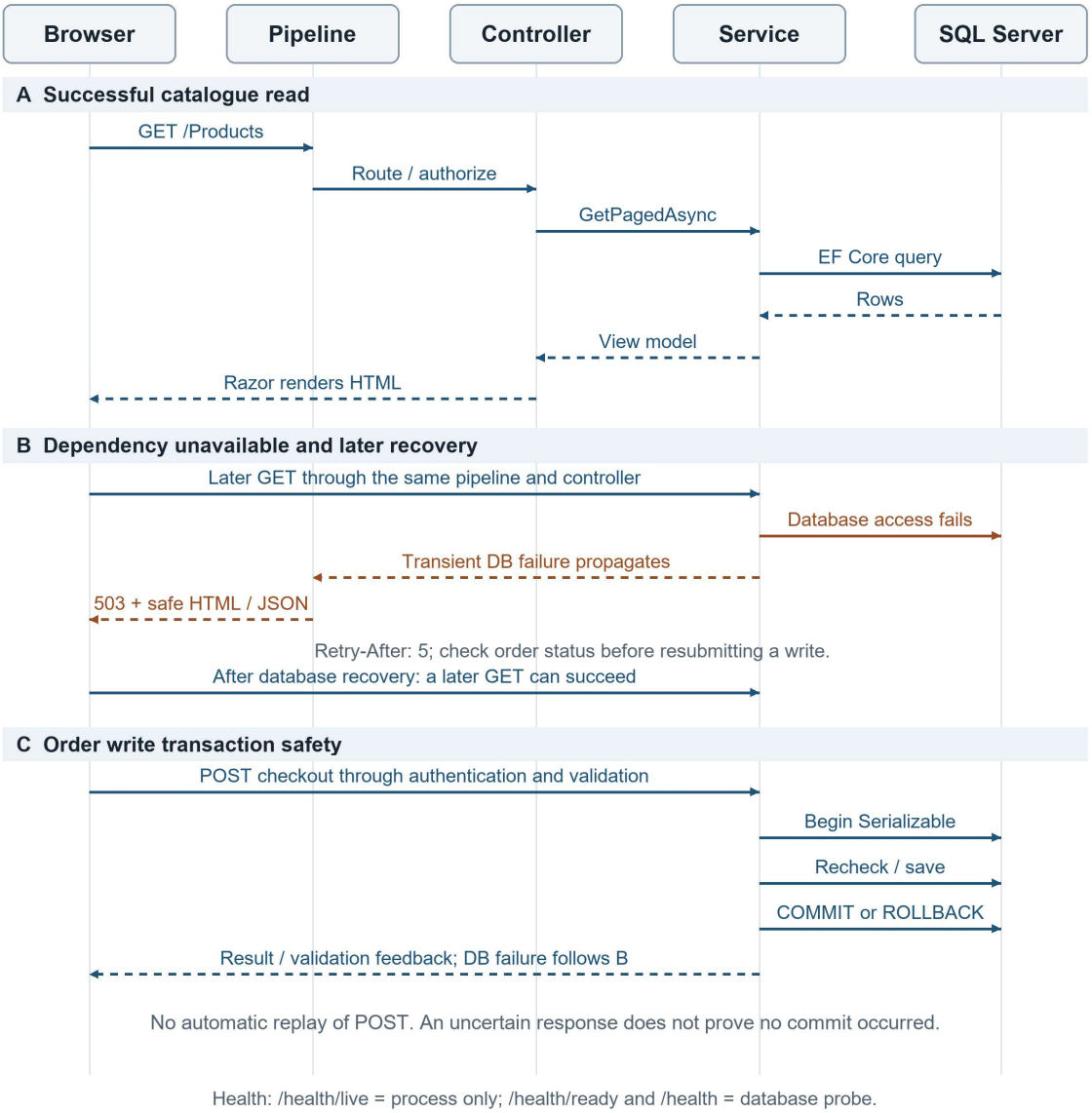


图 3 成功查询 数据故障恢复和订单事务时序

图 3 将正常查询、依赖失败和写入事务分开表示。暂态数据库连接故障在响应尚未开始时由 DatabaseFailureMiddleware 转成安全 HTML 或 JSON，状态为 503，Retry-After 为 5 秒；约束等非暂态错误不会被误报为可重试故障。后续读取可在数据库恢复后重新执行，但服务器不会自动重放下单 POST。连接中断可能发生在提交响应之前，因此用户应先核对订单状态；HTTP 失败不能证明数据库从未提交。[S1、S4]

1.5 数据模型总览

Product 与 Category 为多对多关系，Shop 与 Product 为一对多；ProductSku 保存规格、价格和库存。ProductSpec 与 ProductImage 保存参数和图片。购物车按商品与规格组合区分行，结算时重新加载服务器数据。

Order 与 OrderItem 保存订单及价格、名称和规格快照。SkuId 用于恢复原规格库存；订单还保存双语物流文本、发货预计时间、收货确认和退货物流信息。取消、退款与换货由状态约束控制，不等于调用真实支付或物流系统。[S4]

Review 关联用户、商品和订单。当前评价入口要求订单属于当前用户、状态为 Completed 且包含该商品；服务处理同一订单商品用户组合的重复评价。Favorite 和 DeliveryAddress 分别保存收藏和地址簿。[S3、 S5]

ApplicationUser 扩展 Identity 用户；Coupon 保存优惠规则和用量；EmailVerificationCode 保存验证码及 FailedAttempts 并发字段；HeroSlide 保存双语文案、图片、替代文本、站内链接和展示顺序。验证码仍以明文存储，需要继续评估数据泄漏时的风险。[S2、 S5、 S6]

1.6 关键设计决策

依赖倒置的具体范围：购物车存储、邮件、图片和物流规则都有接口。纯物流策略只接收不同商品的物流事实、是否免邮及显式 UTC 时间，不接收 DbContext 或可跟踪实体。业务服务本身仍依赖 EF Context，不能将局部策略隔离描述为整个应用已完全解耦。

身份与输入边界：Identity、角色授权、防伪令牌、资源归属和服务器价格复核共同保护写入。登录、注册、验证码发送及找回密码共享来源限流；代理转发头只在可信代理链内用于解析来源。限流必须建立在可靠来源上，否则伪造地址就可能产生新计数桶。[S1、 S5]

结构演进：已有迁移保留在 DataAccess；LimitVerificationCodeAttempts 为旧验证码行添加默认值为零的非空计数字段。独立 SQL 场景先运行到旧迁移、插入旧结构数据，再升级并验证数据可继续使用。这比只在空库应用最新结构更接近既有系统升级。[S2、 S8]

交易一致性：OrderService 在 SQL Server 上使用 Serializable 事务，重新校验商品、SKU、数量、库存和优惠券，再保存订单及相关计数。取消或退款只在允许状态恢复一次库存。高隔离级别会增加锁竞争；冲突需反馈给用户，不通过盲目重放写操作掩盖。[S4]

二 架构质量属性 Architectural Quality Attributes

每项属性采用具体场景连接刺激、环境、响应与观察指标。故障反馈会影响可用性和易用性；来源限流保护安全，也可能限制共享网络上的正常用户；纯规则边界既降低修改影响，也使测试更容易。这些联系决定了实现取舍。

2.1 可用性 Availability

场景：应用进程仍运行时，SQL Server 暂时不可连接。目标是区分进程存活与依赖就绪，为请求返回可理解的失败状态，并在依赖恢复后允许后续请求成功。它不要求数据库故障期间继续完成交易。

实现：/health/live 不查询数据库；/health/ready 与兼容入口 /health 使用数据库探针，设置 3 秒取消预算。DatabaseFailureMiddleware 仅在响应未开始时处理已识别的暂态数据库故障，内部细节写入服务器日志，客户端获得 503、请求标识及 Retry-After。约束错误留给既有错误流程，客户端不会从故障反馈看到连接字符串或 SQL 堆栈。[S1]

故障恢复证据：隔离演练的十个步骤全部通过：断库期间 live 返回 200，ready 与目录返回 503，目录带 Retry-After 5 且没有 SQL 细节；数据库 ONLINE 完成后 5168 毫秒内 ready 与目录恢复 200。稳定业务字段前后比较相同，数据库已恢复在线，工具启动的进程已停止。此演练使用隔离数据库，不通过停止共享 SQL 服务影响日常项目。读取恢复、错误码和数据核对是可观察指标；单次本机实验不能推出长期可用率、恢复 SLA 或自动故障转移能力。[S8]

跨属性关系：轮询遇到 503 时保留已有订单内容并提示连接问题，恢复后更新状态，避免空白页面让用户误以为订单丢失。安全的错误消息减少信息泄漏；保留请求标识又为日志定位提供线索。自动重试只适用于确认无副作用的读取，不能代替写入幂等设计。

边界与代价：数据库不可用时依赖它的功能仍不能完成；进程内购物车和限流状态也不能跨重启保留。健康探针本身有数据库访问成本；3 秒是配置预算，实际耗时需运行证据确认。启动迁移失败会记录异常，缺失连接配置则在更早阶段阻止启动。

2.2 可修改性 Modifiability

场景：把下单运费从按不同商品相加改为另一种合单规则，希望不修改控制器、库存更新或事务提交。只有当输入合同仍足够表达新规则时，才应承诺局部修改。[S4]

实现：IOrderShippingPolicy.Quote 接收只读商品物流事实、免邮结果和明确时间，返回不可变 OrderShippingQuote。默认策略负责双语去重、列长度截断、运费合计和发货时效，

既不查库也不保存。OrderService 重新加载商品后映射输入，再应用结果并继续原有事务。

验证：替代策略通过 DI 注入真实下单服务。测试以原运费 3 的商品购买两件，要求保存替代运费 6、订单总额 26、替代快照与时间，库存仍由 5 变为 3。旧实现实际得到 3 而失败，证明测试能识别服务忽略策略的错误；修复后的结果纳入完整回归。[S8]

收益与代价：收费或默认发货时长规则可独立测试，减少修改时需要理解的订单流程。代价是新增接口、不可变对象和映射。代码行数减少不是主要证据；更有意义的是替代策略改变报价而无需改动 IOrderService、控制器与库存事务。

边界：目的地、重量或购买数量尚未进入策略合同，新增这些计费因素仍需调整输入和映射。真实承运商 API 还涉及异步调用、超时和事务时长，不能直接塞进当前同步纯策略。换货的既有快照及固定 48 小时规则不在此次下单策略范围内。

2.3 可测试性 Testability

场景：另一位开发者从仓库重复运行回归，能够知道失败属于规则、前端交互、迁移还是关系数据库并发。输入可构造、结果可观察、环境边界明确，比单纯增加测试数量更有用。

快速层：xUnit 与 EF InMemory 检查服务、控制器和架构边界；纯物流策略用固定时间直接验证结果。Node 使用受控 DOM 检查键盘选择、表单焦点和订单轮询。测试替身缩短反馈时间，但不能证明真实浏览器和 SQL 锁行为。

关系数据库层：仓库中的 tools/verification 以独立环境变量读取本机 master 连接，生成两个全新 GUID 数据库，拒绝日常库和远程连接，并把文件限定在 E 盘。九组场景覆盖全新迁移、旧结构升级、种子、验证码并发、产品分页、库存与退款、末件竞争及双语快照。本轮九组真实 SQL 检查全部通过，JSON 中 completedChecks 为 9、failures 为 0、passed 为 true。[S8]

持续集成：ci.yml 在 Ethan-Yang、HUZHANHAO、main 的 push 与目标 PR 运行 .NET 和 Node，保存 TRX 与 TAP；同时编译 SQL 工具并运行无数据库的隔离检查。真实 SQL 仍由本机显式执行，不把生产密钥交给 CI。报告生成时，新增工作流尚无远程运行结果；后续以 GitHub Actions 记录为准。 本轮总结果为：2026 年 10 月 29 日的 Release 验证中，261 项 .NET 测试与 42 项 Node 测试全部通过；SQL 工具的 17 项隔离检查也全部通过。

2.4 易用性 Usability

场景：用户只用键盘选择筛选项，提交无效表单后找到错误；查看订单期间遇到网络或数据库短暂失败。目标是让控件状态、焦点和反馈与实际操作一致，同时保留用户正在填写的内容。

键盘与表单：自定义下拉支持方向键、Home、End、Enter、Space、Escape 和 Tab，跳过禁用项并同步原生 select。必填的自定义选择器无效时，错误与焦点映射到可操作的触发按钮，防止焦点落在被视觉隐藏的原生 select；其它原生输入保留浏览器验证。[S7]

订单反馈：轮询读取订单 JSON，先判断 HTTP 失败状态，再解析成功数据；失败后显示双语提示并按照 Retry-After 等待。检测到订单变化时触发整页刷新，但存在表单、弹窗或操作焦点时延后，并提供手动刷新入口。连接恢复后移除过期错误；反馈按钮仍获焦时先保留恢复说明，避免焦点突然消失。[S7]

现有任务支持：目录保留关键词、分类、价格和库存筛选及分页；SKU 联动价格、图片、库存与数量上限。购物车和下单服务器复核共同防止用户根据旧页面作出错误购买决定。评价入口校验完成订单，图片被拒绝时保留错误提示并停止此次保存。

验证：本轮 42 项 Node 测试通过。实际浏览器中，End 加 Enter 完成排序选择；订单页在隔离数据库不可用时自动显示错误提示，恢复后自动移除。桌面和移动页面留有截图，未开展参与者实验。 Node 结果能证明脚本在受控输入下的行为；浏览器检查可观察焦点和页面呈现，但两者都不等同正式可用性实验。任务完成率、辅助技术兼容性和真实用户满意度仍需参与者测试。

取舍：轮询改善状态可见性，也增加请求和界面更新。收到数字 Retry-After 时延后自动请求，未收到该提示的网络失败仍沿用每秒轮询。延后刷新保护当前操作，但用户可能稍晚看到最新状态。可用性与易用性通过同一个恢复流程相互支持，状态提示解释了等待原因。

2.5 安全性 Security

场景：未认证请求集中访问账号接口，或攻击者伪造 X-Forwarded-For 试图换来源绕过限制。还需防止并发猜测验证码、越权访问订单和故障页暴露内部信息。安全判断以具体入口和控制为单位，不承诺整个应用绝对无漏洞。

身份与验证码：Identity 使用复杂度规则，五次登录失败锁定五分钟；验证码有 60 秒发送冷却、10 分钟有效期及最多五次预留尝试。并发更新使用 FailedAttempts 并发检查，正确验证码只能成功消费一次；达到上限后即使输入正确也被拒绝。[S1、S5、S8]

来源限流：Login、Register、SendCode、ForgotPassword、Profile、ChangePassword 和 SendProfileEmailCode 的写入口共享 account 策略，按解析后的来源固定窗口每分钟十次、无排队，超额返回 429 与等待时间。已知代理白名单和一层转发限制约束转发头来源。本机 Kestrel 流水线测试确认，改变不可信转发地址不能绕过第十一次请求的 429；可信代理地址可保留独立预算，早期宽松代理配置也被收紧。失败写入只调用一次，JSON 请求无 Accept 头时仍收到结构化错误，内部异常不出现在响应中。 该计数在单进程内，不能宣称跨实例或全站限流。

请求边界：写操作保留防伪令牌、模型与范围校验、站内跳转限制、角色授权和资源归属。订单价格与库存取服务器数据。新安全响应头包括 nosniff、SAMEORIGIN、same-origin Referrer-Policy 及限制 frame-ancestors、base-uri、object-src 的 CSP；该 CSP 未限制全部脚本来源，不能称为完整 XSS 防护。[S1、S5]

输出与失败：普通 Razor 文本使用 HTML 编码，EF LINQ 查询由 provider 参数化。数据库故障页只返回通用说明和请求标识；内部异常保留在服务器日志。管理员富文本仍经 Html.Raw 输出，后续应加入内容白名单清洗。错误细节隐藏与审计可观察性需要同时保留。

登录记忆：RememberedLoginCookieService 现在只保护和保存账号标识，不保存密码。认证 Cookie 与账号预填 Cookie 分工不同；生产认证和 Session Cookie 要求 Secure，账号记忆 Cookie 随实际 HTTPS 设置。部署仍必须正确配置 HTTPS 与 Data Protection 密钥，不能把可解密账号标识当成密码哈希。[S5]

上传边界：图片服务检查扩展名、非空大小、可选 Content-Type、5 MB 上限和图片文件头签名，生成 GUID 文件名并限制删除路径。文件头检查不等于完整图片解码或恶意内容扫描。生产种子不创建开发演示用户，管理员初始化需要明确配置，配置与密钥不可混入报告或测试输出。

取舍与剩余风险：来源限流可能让共享出口的正常用户共同耗尽额度，错误等待提示缓解影响却不能取消限制。验证码仍明文存储，富文本与上传仍有内容边界；分布式限流、支付幂等、验证码哈希化与持续安全审查是后续方向。现有并发和请求测试只证明被覆盖的控制行为。[S5、S8]

2.6 五项属性的共同验证线索

质量属性	场景与作用	主要证据及边界
可用性	区分进程和数据库故障；503 后恢复读取	探针与 HTTP 故障恢复；没有数据库故障转移
可修改性	替换物流计算而保留订单事务	纯策略与替代策略回归；新增计费因素仍需改合同
可测试性	快速回归与真实 SQL 分工，入口可重复	TRX TAP JSON；本地 SQL 不等于 CI 已跑 SQL
易用性	键盘操作 错误焦点 轮询失败与恢复	Node 和浏览器观察；没有真实用户满意度结论
安全性	可信来源限流 验证码尝试上限 安全失败	429 与并发 SQL；仍需富文本和上传内容治理

三 项目反思 Reflection

3.1 当前实现的主要取舍

恢复反馈与交易安全

一次 503 同时涉及可用性、易用性和安全性：状态码供程序判断，双语说明告诉用户下一步，隐藏异常限制信息泄露。但让下单请求自动重试可能产生重复订单。因此本次选择恢复读取和提示核对状态，保留未来通过幂等键解决写入重试的空间。

规则隔离与一致性边界

物流规则属于频繁变化的业务计算，库存事务属于需要统一控制的持久化流程。将两者分开，使报价可替换且可用固定时间测试；若为了降低类文件长度而把事务拆散，反而增加一致性风险。Serializable 提升隔离，但锁竞争仍需在真实负载下观察。

验证速度与证据强度

InMemory 和模拟 DOM 让大多数回归快速执行，SQL 与实际 HTTP 演练补上数据库和协议行为。它们回答不同问题，不能相互冒充。CI 提供重复入口与报告，但配置文件存在并不证明远程运行成功；报告生成时的本机验证与后续 GitHub Actions 记录应分别判断。

3.2 已处理的工程问题

程序集迁移：把业务、数据与模型放入真实类库后，迁移程序集和资源定位都可能变化。本轮保留迁移标识，并用架构测试检查依赖方向、资源解析和迁移发现；真实 SQL 再确认迁移可以执行，而非只验证反射能列出类名。

规格与库存：结算重新校验商品、规格所属关系、启用状态与合计需求。末件并发 SQL 场景要求两次竞争中仅一次成功，库存归零且只留下一个订单明细，避免仅看返回消息就判断没有超卖。

状态恢复：取消和退款按前置状态处理，库存仅恢复一次。重复请求不继续增加商品或 SKU 库存；优惠券 UsedCount 不因取消或退款回退，不能写成所有资源自动恢复。真实 SQL 场景使用多个 DbContext 重新读取状态，减少跟踪缓存掩盖问题的可能。

旧数据升级：只新建空库可能漏掉已有行无法升级的问题。新增迁移场景在旧结构中插入验证码，再升级并消费，直接观察非空默认值和数据保留。它覆盖这一变更路径，仍不能替代部署前备份和完整生产数据迁移演练。

前端语义：键盘移动、确认选择与表单提交是不同动作，错误焦点必须落到可操作控件。轮询更新也要尊重正在编辑的表单。回归针对这些可观察行为，而不是仅比较脚本中是否出现某个属性或函数名。

验证隔离：SQL 工具不能接受任意数据库名，而是自己创建专用名称；master 连接只用于创建新库。产物保留 JSON 和日志，但不记录密码或完整连接串。数据库默认保留供检查，清理应依据报告逐个确认，不能使用匹配宽泛的删除命令。

邮件成功语义：验证码生成与外部投递是两个步骤。只有邮件服务报告成功才显示发送成功，页面不回显验证码；缺配置、网络或供应方错误必须保留失败语义。真实供应商投递仍取决于外部配置，本地服务测试不能替代收件验证。

3.3 MVC 与项目分层的收益和边界

MVC 让请求、视图和输入模型的角色可追踪；项目依赖让编译器能阻止部分反向引用。图 2 展示一次协作，图 1 展示可见依赖，这两者相互补充。代码位置、运行调用和部署单元不应混为同一个层次。

当前 Business 仍引用 DataAccess，部分控制器仍编排 Identity，Models 还包含视图模型与资源。这是一种实际可运行的模块化结构，并非所有依赖都指向纯领域模型的完整 Clean Architecture。报告保留这些边界，便于解释未来修改仍会影响哪里。

独立类库增加项目引用、迁移定位和资源配置的维护成本；纯策略增加输入输出映射。相应收益是更清楚的职责、可替换计算和可自动检查的依赖方向。如果未来需求没有独立部署或扩展压力，不应仅为增加层数再拆分服务。

单进程部署简化课程项目运行与演示，却共享故障、内存 Session 和限流状态。要扩大为多实例，需同步考虑共享会话、密钥、上传存储、限流、监控与数据库容量。类库已经拆开并不表示这些部署问题已解决。

3.4 仍需改进的方向

部署与运行：健康探针和已有 Azure 发布流程提供运维入口，但本轮未部署。后续需在实际部署环境验证 HTTPS、代理信任、密钥和上传持久化，以及重启后的登录与购物车行为，再讨论冗余和恢复目标。

安全与数据：继续评估验证码哈希存储、管理员富文本清洗、图片内容检测和金额列精度。来源限流需要结合真实共享网络行为调节；写入请求若要安全重试，应先设计幂等标识和重复提交的数据库约束，不能直接增加重试次数。

业务与评价：真实支付、承运商及资金退款尚未接入。下一步应扩充关键写事务故障注入、持续负载、真实浏览器端到端和有参与者的可用性测试。可重复 SQL 工具已经落地，后续应扩展其场景，不再把“建立第一个 SQL 集成入口”列为未完成。

3.5 个人反思与贡献 Individual Contribution

以下由每位成员根据实际参与独立填写。项目已具备某项机制，不能证明某位成员亲自完成或掌握；请用提交、问题分析、验证日志及演示记录支持贡献。AI 辅助应区分生成、审查、修改和本人验证，并遵守课程披露要求。

姓名与学号：_____。

我实际完成或验证的任务、对应文件及证据：_____。

我遇到的具体问题、分析过程及处理结果：_____。

我能够独立解释的 MVC 项目分层与五项质量属性，以及仍需复习的内容：_____。

我使用 AI 辅助的范围，以及本人如何审查和验证：_____。

四 代码与验证依据

本报告对应基线提交 4bfdc48 及 2026 年 10 月 29 日的质量改进，最终提交标识可由 Git 历史核对。以下路径均相对于仓库根目录，S1 至 S8 为代码和验证索引。图由 docs/report/build_diagrams.py 重建，正文和证据由同目录源文件生成；结果以本轮实际日志为准。

[S1] OnlineStore/Program.cs；四个应用项目的 csproj；OnlineStore/Infrastructure/ 中的 QualityProtectionExtensions、DatabaseFailureMiddleware、DatabaseReadinessCheck、FailureResponse、SecurityResponseHeadersMiddleware。

[S2] OnlineStore.DataAccess/Data/ApplicationDbContext.cs、DbInitializer.cs；OnlineStore.DataAccess/Migrations/，尤其 20261029140000_LimitVerificationCodeAttempts.cs；OnlineStore.Models/Models/。

[S3] OnlineStore/Controllers/ProductsController.cs、ReviewsController.cs；OnlineStore.Business/Services/ProductService.cs、ReviewService.cs；OnlineStore.Models/ViewModels/ProductListViewModel.cs；OnlineStore/Views/Products/Index.cshtml。

[S4] OnlineStore/Controllers/OrdersController.cs；OnlineStore.Business/Services/OrderService.cs、CartService.cs、CouponCalculator.cs 及 Shipping/；OnlineStore.Models/Models/Order.cs、OrderItem.cs；docs/quality-modifiability.md。

[S5] OnlineStore/Controllers/AccountController.cs；OnlineStore.Business/Services/VerificationCodeService.cs、RememberedLoginCookieService.cs、ImageStorageService.cs、BrevoApiEmailService.cs、SmtpEmailService.cs；OnlineStore/Views/Products/_ProductStory.cshtml。

[S6] OnlineStore/Areas/Admin/Controllers/HeroSlidesController.cs；OnlineStore.Business/Services/HeroSlideService.cs；OnlineStore/ViewComponents/HeroCarouselViewComponent.cs；OnlineStore/Views/Home/_HeroCarousel.cshtml。

[S7] OnlineStore/wwwroot/js/site.js、customer-order-updates.js；OnlineStore/Views/Orders/MyOrders.cshtml；OnlineStore.Tests/Frontend/custom-select.test.cjs、customer-order-updates.test.cjs 及其余前端测试。

[S8] OnlineStore.Tests/Architecture/LayerBoundaryTests.cs；Controllers/QualityPipelineTests.cs；Services/OrderShippingPolicyTests.cs、OrderShippingPolicyIntegrationTests.cs 及既有回归；tools/verification/；.github/workflows/ci.yml；docs/verification.md。本轮证据位置：docs/quality-evidence.md；docs/evidence/recovery-run-2/recovery-verification.json；原始结果位于 E:/DMU-Learning/.tmp/quality-uplift-20261029 下的 test-results/quality-final-261.trx、frontend-final.log、sql-repository-run-2/sql-verification.json。